Phase 9 Real-World CVRP Solver Evolution Protocol

This document scopes the next bounded phase after the phase-8 adaptive-portfolio result.

The thesis-level question is:

> *Can the LLM-evolution loop autonomously synthesize useful solver logic for a real-world combinatorial optimization benchmark, given only a problem specification, parser, validator, scoring function, and training instances?*

This phase studies solver-code evolution, not portfolio selection.

Dataset Choice

Phase 9 uses the official CVRPLIB Uchoa X benchmark family as the primary real-world target.

Why this choice is the most practical next step:

- it is a standard public benchmark with official instance and solution files
- it is objectively scorable against published best-known solutions
- it is much messier than symmetric TSP because feasibility depends on route completeness and vehicle capacity, not only route cost
- the repository already has CVRP parsing and scoring infrastructure that can be extended cleanly into a whole-solver track
- it avoids the extra parser and feasibility complexity of VRPTW as a first real-world solver-evolution phase

Primary sources:

- CVRPLIB official catalog: CVRPLIB Instances (https://galgos.inf.puc-rio.br/cvrplib/en/instances)
- Uchoa et al. 2017: New benchmark instances for the capacitated vehicle routing problem (https://doi.org/10.1016/j.ejor.2016.08.012)
- DIMACS CVRP challenge overview: DIMACS Capacitated VRP (https://dimacs.rutgers.edu/programs/challenge/vrp/cvrp/)

The bounded benchmark subset for this phase is a curated moderate-size slice of the X set. The intent is to expose real feasibility logic while keeping official runs affordable, while also covering both two-cluster and grid-like regimes under the project descriptor basis.

Committed train instances:

- X-n101-k25
- X-n110-k13
- X-n125-k30
- X-n134-k13
- X-n157-k13
- X-n214-k11

Committed held-out instances:

- X-n176-k26
- X-n190-k8

- X-n223-k34
- X-n247-k50
- X-n275-k28

Under the current descriptor classifier, this split yields train coverage of five two_cluster_bottleneck instances plus one grid_like instance, and held-out coverage of three two_cluster_bottleneck instances plus two grid_like instances.

Parser, Validator, and Scorer

Phase 9 requires a strict CVRP pipeline:

- parse official .vrp instance files
- parse official solution files
- validate that every customer is served exactly once
- validate that no route exceeds vehicle capacity
- compute total route cost with TSPLIB Euclidean rounding
- compute objective gap to the official best-known solution

For this bounded X-instance phase, route count is *not* enforced as a feasibility constraint. This follows the standard unrestricted-route CVRP interpretation used in the DIMACS CVRP challenge, which explicitly accepts solutions with more routes than the k encoded in the instance name. The vehicle_count_hint is still logged and passed to the solver as contextual information, but it is not treated as a hard validator rule in this phase.

Scoring outputs for each solver are:

- feasibility rate
- mean penalized gap
- mean feasible-instance objective gap
- runtime
- robustness across held-out structure families

The phase uses a penalized-gap selection score so infeasible solvers cannot win by returning partial or broken solutions.

Per-instance solver-worker timeouts are also treated as failed evaluations under this penalized-gap rule; they should not abort a whole suite run. The model-call timeout and solver-worker timeout are tracked separately. In the official bounded suite, the solver-worker timeout is larger than observed baseline runtimes but still finite (60 seconds) so pathological generated code is penalized quickly rather than dominating wall-clock time for the whole campaign.

Baselines

The bounded official baseline set is:

1. nearest-neighbor constructive
2. Clarke-Wright savings
3. regret-insertion plus simple local search

These are built into the phase-9 package and do not depend on external solvers.

OR-Tools is documented as an optional external reference, not a required core condition, because it is not bundled in the current environment and the main claim does not require it.

What The LLM Is Allowed To Evolve

The generated program must define:

```
def solve_cvrp(instance):
 return routes
```

Where routes is a list of routes and each route is a list of zero-based customer node ids, excluding the depot.

The solver may evolve:

- constructive heuristics
- savings-style merges
- insertion order and insertion criteria
- repair logic for incomplete or overloaded routes
- intra-route local search
- inter-route relocate or swap logic
- destroy/repair cycles
- restart policies
- instance-conditioned control based on descriptors

The solver may not use:

- imports
- hidden external solvers
- file or network access
- hard-coded instance-name rules
- non-deterministic randomness

This search object is more open than the phase-5 scaffold track, while remaining bounded enough to keep the resulting code interpretable and auditable.

Invalid or fallback generations are still logged for diagnostic purposes, but they are not eligible to replace the incumbent solver in the evolutionary loop. Only generations that pass the bounded code-validation path may be accepted. The evolutionary prompt is mutation-oriented rather than rewrite-oriented: the model sees the incumbent solver and is instructed to make a few focused edits instead of replacing the whole design from scratch. The runtime also applies deterministic tail-salvage to oversized or obviously truncated submissions before falling back to a full repair call. This keeps the search close to local heuristic mutation instead of unconstrained whole-program regeneration at every epoch.

Inputs Available To The Solver

The runtime instance payload includes:

- customer ids
- depot index
- demands
- vehicle capacity
- vehicle-count hint
- coordinates

- full distance matrix
- nearest-neighbor candidate lists
- derived descriptors such as size, demand pressure, clusteredness, corridor score, bottleneck score, and trap score

This gives the solver enough information to build constructive, repair, and restart logic without depending on hidden benchmark metadata.

To reduce direct benchmark-identifier overfitting, the runtime payload does not expose instance names, and the generation prompt refers to anonymous training-case summaries rather than named benchmark instances.

Evaluation Discipline

- The evolutionary loop trains only on the training subset.
- Held-out instances are evaluated only after freezing the final accepted solver.
- Official reporting should emphasize feasibility first, then objective gap, runtime, and family robustness.
- The project should not claim state-of-the-art performance.

The intended claim shape is narrower:

> *given only the specification, validator, scoring function, and training instances, the LLM-evolution loop can progress from naive or brittle solvers toward feasible, competitive, interpretable solver logic on held-out real-world CVRP benchmarks.*

Official Bounded Suite

The default official suite contains four conditions:

1. nearest-neighbor constructive baseline
2. Clarke-Wright savings baseline
3. regret-insertion plus local-search baseline
4. solver evolution

That keeps the phase bounded and directly answers the supervisor's request without turning phase 9 into another large ablation tree.

Operational Entry Points

- prepare the benchmark with prepare_cvrp_phase9_benchmarks.py
- run the suite with run_cvrp_phase9_suite.py
- aggregate repeated runs with aggregate_cvrp_phase9_runs.py
- use configs/cvrp_phase9_suite/RUNBOOK.md for smoke and official commands
- the official evidence readout is tracked in docs/PHASE_9_REAL_WORLD_CVRP_SOLVER_EVOLUTION_RESULTS_2026-05-21.md